

机器人开发实践实验手册

一. win系统所需环境

1.1 安装Git

下载并安装 Git for Windows, 选择适合你系统架构的版本。

1.2 安装 Anaconda

可以到Anaconda官网下载, 安装完, 设置电脑的系统变量, 在变量中找到 Path, 点击 编辑, 将Conda 安装目录下的 Scripts 路径

例如: 添加安装目录 (如C:\ProgramData\Anaconda3\Scripts) 到Path中

二. Lerobot环境配置

1. 从课程github仓库中拉取lerobot代码

```
git clone https://github.com/tyzjbclg/ecnu_robotlab_lerobot.git
```

2. 右键Anaconda Prompt以管理员身份打开然后创建conda环境

```
conda create -y -n lerobot python=3.10
```

3. 激活conda环境

```
conda activate lerobot
```

4. 进入到从仓库拉取下来的代码目录

```
cd 代码保存的目录
```

5. 安装Feetech SDK库文件 (深圳飞特舵机 <https://www.feetechrc.com>)

```
pip install -e "[feetech]"
```

6. 在目录下查找主从臂接口id

```
python lerobot/scripts/find_motors_bus_port.py
```

运行代码后会输出如下内容

```
(lerobot) E:\zhujiao\lerobot-user-michel-aractingi-2025-06-02-docs-for-hil-serl>python lerobot/scripts/find_motors_bus_port.py
Finding all available ports for the MotorsBus.
Ports before disconnecting: ['COM5', 'COM4', 'COM6', 'COM3']
Remove the USB cable from your MotorsBus and press Enter when done.
```

此时拔掉从臂usb接口按下回车会输出从臂的com id

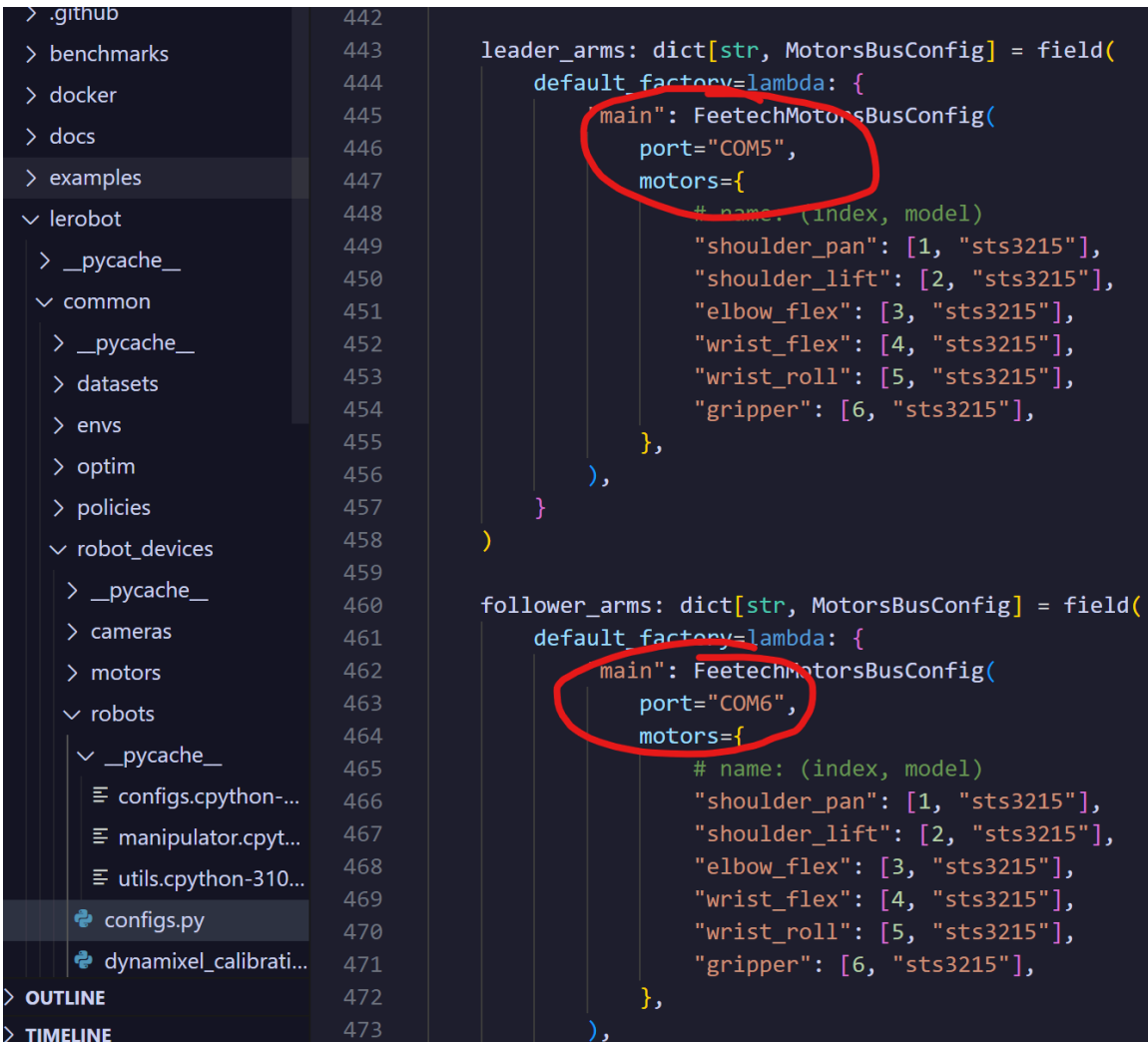
```
ort.py
Finding all available ports for the MotorsBus.
Ports before disconnecting: ['COM5', 'COM4', 'COM6', 'COM3']
Remove the USB cable from your MotorsBus and press Enter when done.

The port of this MotorsBus is 'COM6'
Reconnect the USB cable.
```

相同的操作同理也对主臂进行一次，此时我们记录下主臂与从臂的COM id

然后找到 路径lerobot-main\lerobot\common\robot_devices\robots的config.py文件

修改446和463行的主臂和从臂的接口ID



```
> .github 442
> benchmarks 443
> docker 444
> docs 445
> examples 446
> lerobot 447
  > __pycache__ 448
  > common 449
    > __pycache__ 450
    > datasets 451
    > envs 452
    > optim 453
    > policies 454
    > robot_devices 455
      > __pycache__ 456
      > cameras 457
      > motors 458
      > robots 459
        > __pycache__ 460
        ≡ configs.cpython-... 461
        ≡ manipulator.cpyt... 462
        ≡ utils.cpython-310... 463
        📄 configs.py 464
        📄 dynamixel_calibrati... 465
> OUTLINE 466
> TIMELINE 467

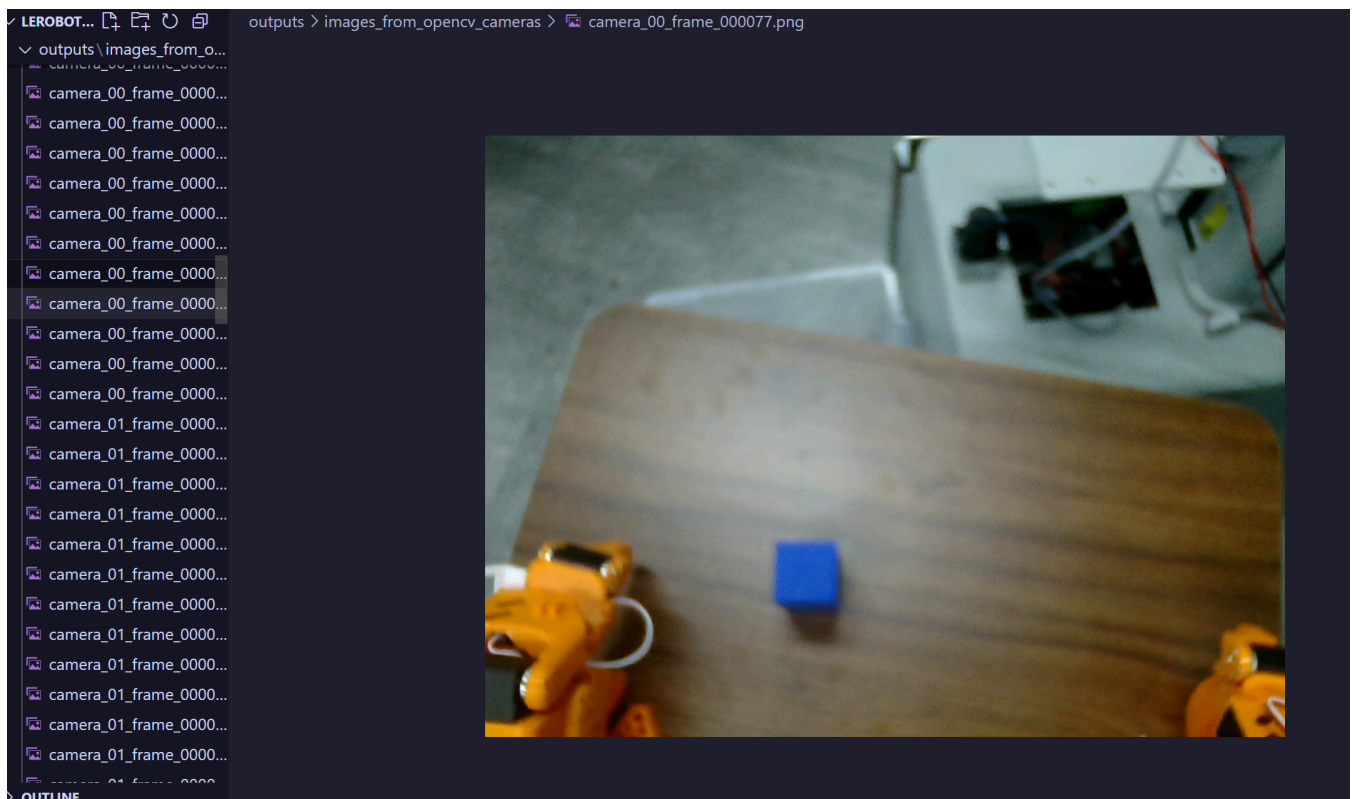
442 leader_arms: dict[str, MotorsBusConfig] = field(
443     default_factory=lambda: {
444         "main": FeetechMotorsBusConfig(
445             port="COM5",
446             motors={
447                 # name: (index, model)
448                 "shoulder_pan": [1, "sts3215"],
449                 "shoulder_lift": [2, "sts3215"],
450                 "elbow_flex": [3, "sts3215"],
451                 "wrist_flex": [4, "sts3215"],
452                 "wrist_roll": [5, "sts3215"],
453                 "gripper": [6, "sts3215"],
454             },
455         ),
456     },
457 )
458
459 follower_arms: dict[str, MotorsBusConfig] = field(
460     default_factory=lambda: {
461         "main": FeetechMotorsBusConfig(
462             port="COM6",
463             motors={
464                 # name: (index, model)
465                 "shoulder_pan": [1, "sts3215"],
466                 "shoulder_lift": [2, "sts3215"],
467                 "elbow_flex": [3, "sts3215"],
468                 "wrist_flex": [4, "sts3215"],
469                 "wrist_roll": [5, "sts3215"],
470                 "gripper": [6, "sts3215"],
471             },
472         ),
473     },
474 )
```

7.相机配置

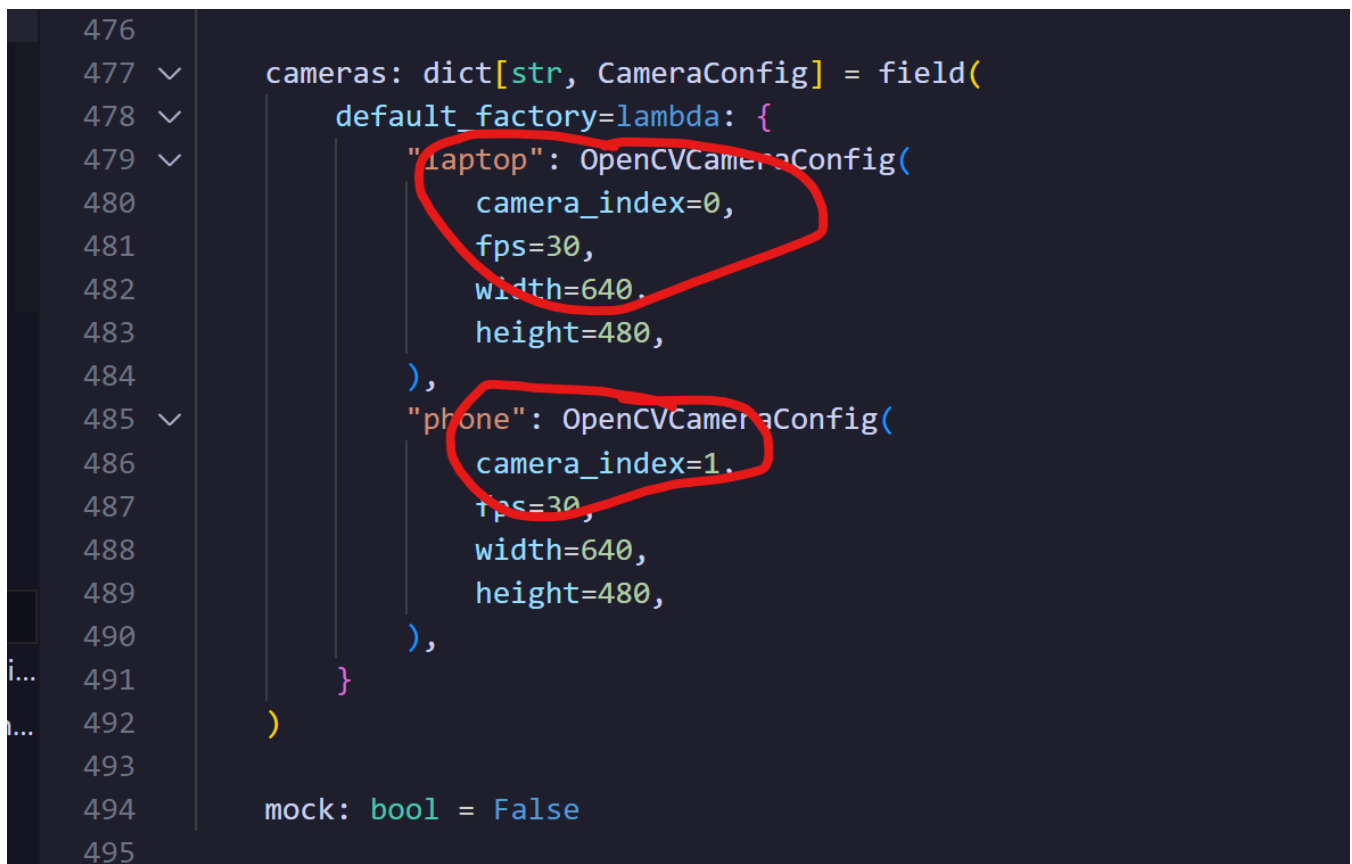
执行如下代码，会调用电脑所连接的所有相机进行拍照

```
python lerobot/common/robot_devices/cameras/opencv.py --images-dir
outputs/images_from_opencv_cameras
```

拍摄的照片会以相机ID命名存储在outputs/images_from_opencv_cameras中，camera后面的数字就是id



然后修改lerobot-main\lerobot\common\robot_devices\robots的config.py文件中的第480和486行 还有257行和263行，改成自己的相机id，laptop为顶视相机，phone为夹爪相机



然后修改lerobot-main\lerobot\common\robot_devices\cameras\opencv.py 的333行，将第333行代码注释掉之后再下面添加如下代码。

```
#self.camera = cv2.VideoCapture(camera_idx, backend) #把原来的注释掉
self.camera = cv2.VideoCapture(camera_idx)
self.camera.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter.fourcc("M", "J", "P", "G"))
self.camera.set(cv2.CAP_PROP_FPS, 60)
```

8.在conda环境中安装pytorch

首先查看自己电脑配置的显卡信息以及需要安装的版本

```
nvidia-smi
```

```

Tue Feb  3 19:47:16 2026
+-----+
| NVIDIA-SMI 572.97              | Driver Version: 572.97      | CUDA Version: 12.8 |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                | Driver-Model  | Bus-Id  | Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          | Pwr:Usage/Cap |          | Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
|   0  NVIDIA GeForce RTX 5060 ...  | WDDM          | 00000000:64:00:0  | On      |         N/A         |
| N/A   43C    P5              | 7W / 70W      | 1824MiB / 8151MiB |         0%   Default |
+-----+-----+-----+-----+-----+-----+
| Processes: |
| GPU   GI   CI          PID    Type   Process name                      | GPU Memory |
|=====+=====+=====+=====+=====+=====+
|   0   N/A   N/A              8664      G-S      C:\Program Files\H...              | N/A         |
+-----+-----+-----+-----+-----+-----+

```

这里可以看到我的显卡是5060，需要安装的CUDA版本是12.8，则允许如下命令安装pytorch，每个人的版本号不一样需要将最后的cu改成自己显卡的cuda版本

```
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu128
```

三. 对lerobot进行遥操作

首先执行如下代码

```
python lerobot/scripts/control_robot.py --robot.type=so101 --robot.cameras="{}" --
control.type=calibrate --control.arms=["main_follower"]
```

然后会输出如下内容：

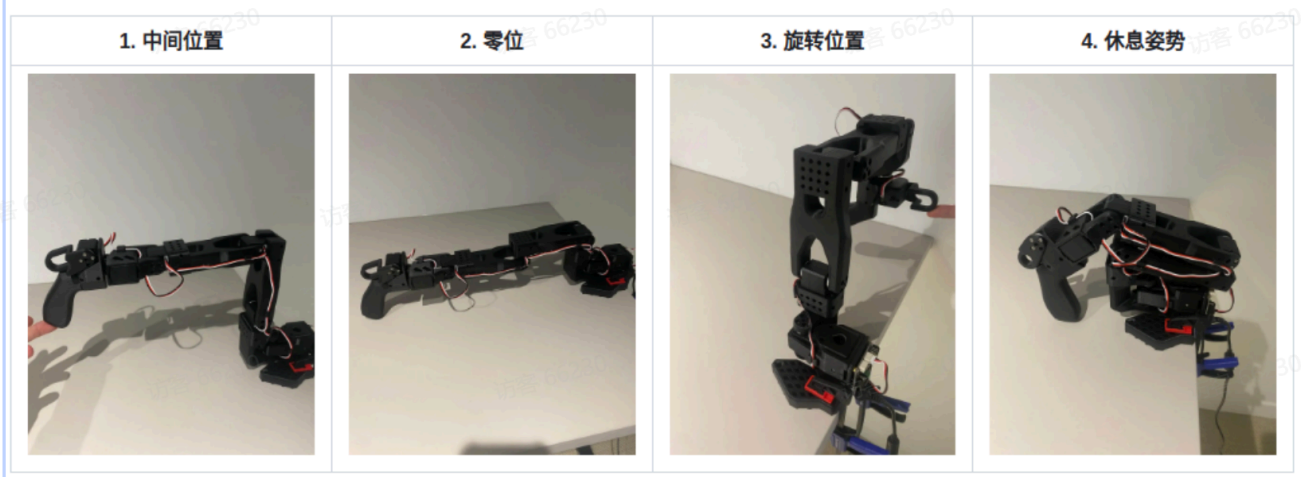
```
Running calibration of so101 main follower...
Please move the robot to the new middle position for calibration, then press Enter...
```

然后以此按照下图四个图依次手动摆好从臂的位置，摆好一个位置按一下回车



手动校准引导臂

您还需要按顺序将引导臂移动到以下位置：



完成如下图，注意：完成从臂的标注后会自动开始进行主臂的标注

```
Running calibration of so101 main follower...
Please move the robot to the new middle position for calibration, then press Enter...

Move arm to zero position
See: https://raw.githubusercontent.com/huggingface/lerobot/main/media/so101/follower_zero.webp
Press Enter to continue...

Move arm to rotated target position
See: https://raw.githubusercontent.com/huggingface/lerobot/main/media/so101/follower_rotated.webp
Press Enter to continue...

Move arm to rest position
See: https://raw.githubusercontent.com/huggingface/lerobot/main/media/so101/follower_rest.webp
Press Enter to continue...

Calibration is done! Saving calibration file '.cache\calibration\so101\main_follower.json'
Missing calibration file '.cache\calibration\so101\main_leader.json'

Running calibration of so101 main leader...
Please move the robot to the new middle position for calibration, then press Enter...

Move arm to zero position
See: https://raw.githubusercontent.com/huggingface/lerobot/main/media/so101/leader_zero.webp
Press Enter to continue...

Move arm to rotated target position
```

动作标定完成后，我们开始进行遥操作，指令如下：

```
python lerobot/scripts/control_robot.py --robot.type=so101 --control.type=teleoperate --
control.display_data=true
```

执行完成后，我们用手操作主臂，从臂就会跟着主臂做一样的操作

四. 数据采集

执行下列代码进行数据采集，control.warmup_time_s=5是准备时间为5s，control.episode_time_s=15是数据录制时间为15s，control.reset_time_s=5是重置环境的时间，control.num_episodes=50是一次性采集数据的数量。

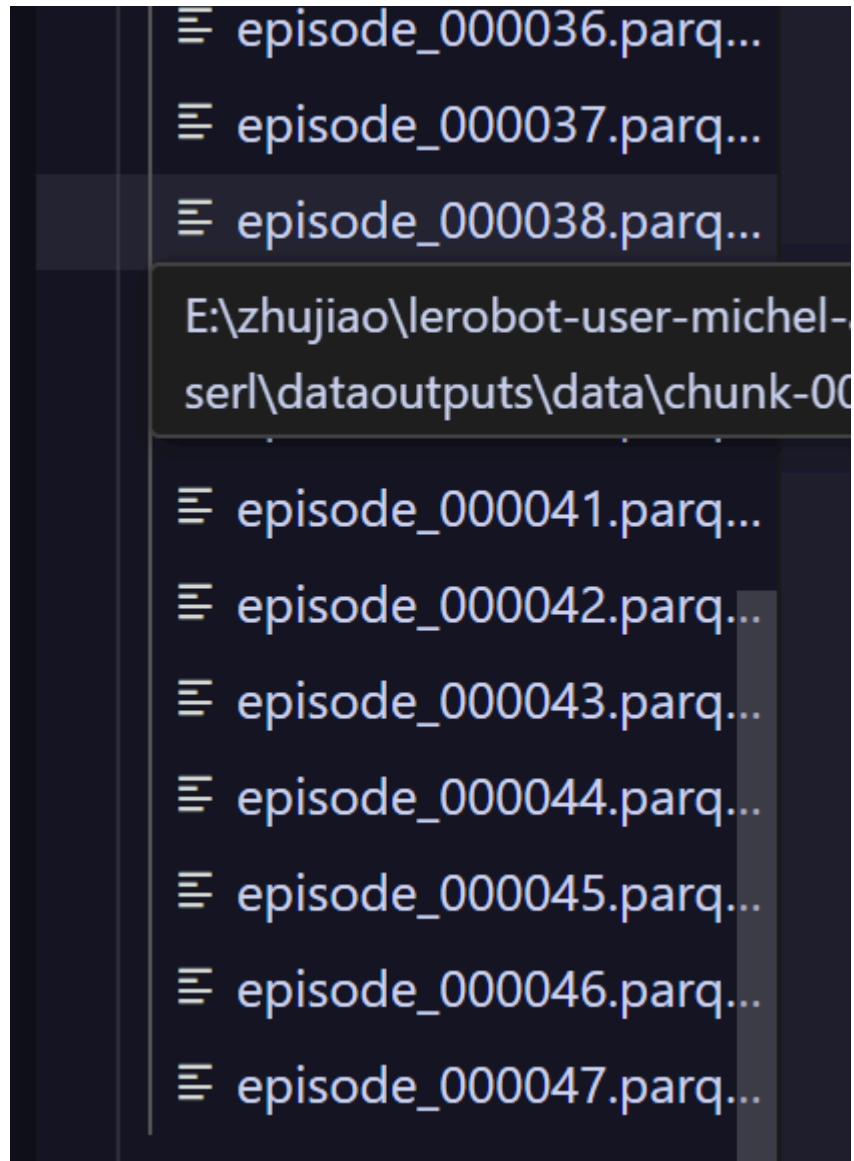
```
python lerobot/scripts/control_robot.py --robot.type=so101 --control.type=record --
control.fps=30 --control.single_task="Grasp a lego block and put it in the bin." --
control.repo_id=myuser/so101_lego_grasp --control.root=dataoutputs --control.warmup_time_s=5
--control.episode_time_s=15 --control.reset_time_s=5 --control.num_episodes=50 --
control.display_data=true --control.push_to_hub=false --control.video=false
```

如果想要再数据集后增采集，需要在后面增加参数 control.resume=true

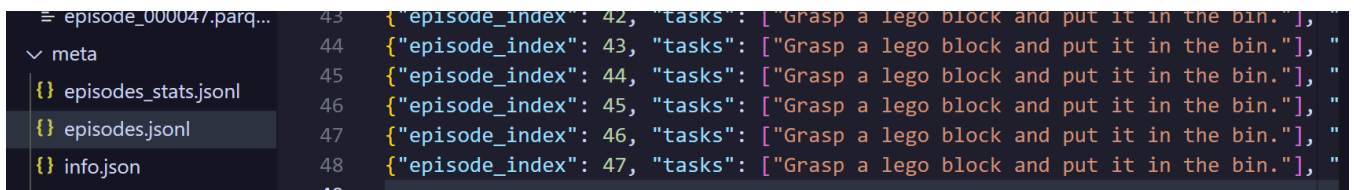
```
python lerobot/scripts/control_robot.py --robot.type=so101 --control.type=record --
control.fps=30 --control.single_task="Grasp a lego block and put it in the bin." --
control.repo_id=myuser/so101_lego_grasp --control.root=dataoutputs --control.warmup_time_s=5
--control.episode_time_s=15 --control.reset_time_s=5 --control.num_episodes=10 --
control.display_data=true --control.push_to_hub=false --control.video=false --
control.resume=true
```

如果想要删除废弃的数据，则需要删除这些内容并且更改相关参数：

1. 删除废弃的数据



2.删除episodes_stats.jsonl和episodes.jsonl里的json对应的json文件



3.修改info.json里的"total_episodes","total_frames",和splits里的"train"参数


```

robot_type : so101 ,
"total_episodes": 48,
"total_frames": 20922,
"total_tasks": 1,
"total_videos": 0,
"total_chunks": 1,
"chunks_size": 1000,
"fps": 30,
"splits": {
    "train": "0:48"
}

```

运行以下命令来验证数据集,如果终端打印数据集加载成功共多少个episodes,则表明数据加载成功。这里会跑得比较慢。

```

python -c "from lerobot.common.datasets.lerobot_dataset import LeRobotDataset; ds =
LeRobotDataset('myuser/so101_lego_grasp', root='dataoutputs'); print(f'✅ 数据集加载成功! 共
{ds.num_episodes} 个 episodes, {ds.num_frames} 帧')"

```

五. 模型训练

运行以下命令来进行模型训练: steps=10000是训练步数,步数越多越好,但是考虑到训练时间成本,我们就设置为10000步, batch_size值参考自己的显存,我的显存是8g,数值是16,大家可以根据自己的显存动态调整, policy.n_action_steps是预测未来动作的步数,大家也可以根据自己的算了自行调整

```

python lerobot/scripts/train.py --dataset.repo_id="" --dataset.root=dataoutputs --
policy.type=act --output_dir=outputs/train/act_lego_grasp --job_name=act_lego_grasp --
policy.device=cuda --num_workers=2 --batch_size=16 --policy.n_obs_steps=1 --
policy.n_action_steps=10 --policy.chunk_size=10 --steps=10000

```

参数说明:

dataset.repo_id=myuser/so101_lego_grasp: 你的数据集
--dataset.root=dataoutputs: 数据集路径
--policy.type=act: 使用 ACT 策略
--output_dir=outputs/train/act_lego_grasp: 训练输出目录
--policy.device=cuda: 使用 GPU 训练 (如果没有 GPU, 改为 cpu)

六. 对训练完的模型在真机上进行推理验证

```
python lerobot/scripts/control_robot.py --robot.type=so101 --control.type=record --
control.fps=30 --control.single_task="Grasp a lego block and put it in the bin." --
control.repo_id=myuser/eval_act_lego_grasp --control.root=realoutputs --
control.num_episodes=10 --control.warmup_time_s=2 --control.episode_time_s=30 --
control.reset_time_s=10 --
control.policy.path=outputs/train/act_lego_grasp/checkpoints/last/pretrained_model --
control.push_to_hub=false --control.video=false
```